Mini Project

Title: Analyse a Real-World Dataset, Build an Interactive Dashboard and Present Actionable Insights with Ethical Considerations

Enrollment No.: 250843116002

Aim:

To analyse a real-world business dataset (online food delivery), design and develop an interactive dashboard using Dash/Plotly for business reporting, and present actionable insights while addressing ethical considerations.

Problem Statement & Dataset

An online food delivery platform serving customers in four major cities of India (Mumbai, Delhi, Bengaluru and Hyderabad) wants to understand its 2024 performance. The dataset used here is a simulated food delivery dataset (200 orders, 13 columns) modelled on real-world patterns, such as festive-season and monsoon peaks, discount-driven losses and slow deliveries. It contains no real customer information; customer IDs are anonymised codes.

The analysis answers three business questions:

- Which cities, cuisines and dishes drive revenue and profit?
- How do discounts and delivery delays affect profitability and customer ratings?
- When does demand peak during the year?

Dataset Description

Column	Description
Order_ID	Unique order number
Order_Date	Date of the order (2024)
Customer_ID	Anonymised customer code
Customer_Type	New / Regular / Premium
City	Delivery city (Mumbai, Delhi, Bengaluru, Hyderabad)
Cuisine, Dish	Menu hierarchy (6 cuisines, 10 dishes)
Quantity	Units of the dish in the order
Discount	Discount given (0 to 0.30)
Order_Value, Profit	Order value and platform profit in INR
Delivery_Time	Time from order to delivery, in minutes
Rating	Customer rating of the order (1 to 5)

Step 1: Install libraries

```
# Install the required libraries
!pip install -q dash plotly pandas numpy
```

Step 2: Import Required libraries

```
# Import Pandas and NumPy
import pandas as pd
import numpy as np
# Import Plotly Express
import plotly.express as px
# Import Dash components
from dash import Dash, dcc, html, Input, Output
```

```
# Display settings for tables
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)
```

Step 3: Create the dataset

```
# Create a simulated food delivery dataset (India, year 2024)
np.random.seed(7)
n = 200
# Menu: (cuisine, unit price in INR, base profit margin)
menu = {
    "Butter Chicken Combo": ("North Indian", 520, 0.16),
    "Paneer Thali": ("North Indian", 380, 0.18),
    "Masala Dosa Combo": ("South Indian", 220, 0.22),
    "Idli Vada Platter": ("South Indian", 160, 0.24),
    "Veg Hakka Noodles": ("Chinese", 280, 0.20),
    "Burger Meal": ("Fast Food", 300, 0.19),
    "Pizza Large": ("Fast Food", 650, 0.15),
    "Chocolate Brownie": ("Desserts", 180, 0.30),
    "Ice Cream Tub": ("Desserts", 250, 0.26),
    "Salad Bowl": ("Healthy", 340, 0.21),
}
cities = ["Mumbai", "Delhi", "Bengaluru", "Hyderabad"]
city = np.random.choice(cities, n, p=[0.30, 0.27, 0.25, 0.18])
dish = np.random.choice(list(menu), n,
                        p=[0.12, 0.10, 0.12, 0.08, 0.10, 0.12, 0.14, 0.06, 0.06, 0.10])
cuisine = [menu[d][0] for d in dish]
price = np.array([menu[d][1] for d in dish])
margin = np.array([menu[d][2] for d in dish])
# Festive season (Oct-Dec) and monsoon (Jul-Aug) receive more orders
month_w = np.array([8, 7, 8, 7, 7, 8, 10, 10, 9, 13, 12, 14], dtype=float)
month = np.random.choice(np.arange(1, 13), n, p=month_w / month_w.sum())
day = np.random.randint(1, 28, n)
order_date = pd.to_datetime(dict(year=2024, month=month, day=day))
quantity = np.random.randint(1, 5, n)
discount = np.random.choice([0, 0.05, 0.10, 0.20, 0.30], n,
                            p=[0.30, 0.25, 0.20, 0.15, 0.10])
# Delivery time (minutes): city traffic + monsoon/festive rush + random variation
base_time = pd.Series(city).map({"Mumbai": 36, "Delhi": 33,
                                 "Bengaluru": 40, "Hyderabad": 31}).values
rush = np.where(np.isin(month, [7, 8, 10, 11, 12]), 4, 0)
delivery_time = np.clip(np.random.normal(base_time + rush, 8, n),
                        15, 80).round(0).astype(int)
# Rating drops as delivery gets slower
rating = np.clip(4.6 - 0.03 * np.maximum(delivery_time - 30, 0)
                 + np.random.normal(0, 0.3, n), 1, 5).round(1)
order_value = (quantity * price * (1 - discount)).round(0)
# Profit = margin - discount cost; late orders (>45 min) lose 8% to refunds
profit = (order_value * (margin - 0.7 * discount + np.random.normal(0, 0.03, n))
          - np.where(delivery_time > 45, 0.08 * order_value, 0)).round(0)
df = pd.DataFrame({
    "Order_ID": [f"ORD{1000 + i}" for i in range(n)],
    "Order_Date": order_date,
    "Customer_ID": [f"C{np.random.randint(1, 181):04d}" for _ in range(n)],
    "Customer_Type": np.random.choice(["New", "Regular", "Premium"],
                                      n, p=[0.35, 0.45, 0.20]),
    "City": city,
    "Cuisine": cuisine,
    "Dish": dish,
    "Quantity": quantity,
    "Discount": discount,
    "Order_Value": order_value.astype(int),
    "Profit": profit.astype(int),
    "Delivery_Time": delivery_time,
    "Rating": rating,
}).sort_values("Order_Date").reset_index(drop=True)
df["Order_ID"] = [f"ORD{1000 + i}" for i in range(n)]
# Simulate real-world data-quality issues (missing values and duplicates)
df.loc[[8, 35, 70, 110, 150, 185], "Customer_Type"] = np.nan
df = pd.concat([df, df.iloc[[10, 50, 120, 180]]], ignore_index=True)
print(df.head(10))
print("\nShape:", df.shape)
```

Output:

```
  Order_ID Order_Date Customer_ID Customer_Type       City       Cuisine                Dish  Quantity  Discount  Order_Value  Profit  Delivery_Time  Rating
0  ORD1000 2024-01-02       C0166       Regular     Mumbai  South Indian     Masala Dosa Combo         1      0.20          176      10             36     4.3
1  ORD1001 2024-01-04       C0083       Regular      Delhi  South Indian     Idli Vada Platter         4      0.05          608     129             32     4.4
2  ORD1002 2024-01-05       C0017           New  Bengaluru      Desserts     Chocolate Brownie         1      0.05          171      45             37     4.7
3  ORD1003 2024-01-08       C0179       Regular  Hyderabad      Desserts     Chocolate Brownie         1      0.10          162      38             36     4.7
4  ORD1004 2024-01-13       C0117       Regular  Bengaluru  North Indian  Butter Chicken Combo         1      0.05          494      -2             50     3.8
5  ORD1005 2024-01-15       C0150       Premium  Bengaluru       Chinese     Veg Hakka Noodles         2      0.00          560     114             34     4.5
6  ORD1006 2024-01-15       C0090       Premium     Mumbai  South Indian     Idli Vada Platter         2      0.30          224      -3             43     4.1
7  ORD1007 2024-01-16       C0064       Regular      Delhi      Desserts         Ice Cream Tub         3      0.30          525       4             38     3.9
8  ORD1008 2024-01-16       C0033           NaN      Delhi       Chinese     Veg Hakka Noodles         4      0.20          896      64             22     4.3
9  ORD1009 2024-01-25       C0065       Regular      Delhi  South Indian     Masala Dosa Combo         3      0.20          528      58             37     4.5
Shape: (204, 13)
```

Step 4: Inspect the dataset

```python
# Inspect the structure of the dataset
df.info()
print()
# Statistical summary of numeric columns
print(df[["Quantity", "Discount", "Order_Value", "Profit",
          "Delivery_Time", "Rating"]].describe().round(2))
```

Output:

```
<class 'pandas.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Order_ID       204 non-null    str
 1   Order_Date     204 non-null    datetime64[us]
 2   Customer_ID    204 non-null    str
 3   Customer_Type  198 non-null    str
 4   City           204 non-null    str
 5   Cuisine        204 non-null    str
 6   Dish           204 non-null    str
 7   Quantity       204 non-null    int64
 8   Discount       204 non-null    float64
 9   Order_Value    204 non-null    int64
 10  Profit         204 non-null    int64
 11  Delivery_Time  204 non-null    int64
 12  Rating         204 non-null    float64
dtypes: datetime64[us](1), float64(2), int64(4), str(6)
memory usage: 20.8 KB
       Quantity  Discount  Order_Value  Profit  Delivery_Time  Rating
count    204.00    204.00       204.00  204.00         204.00  204.00
mean       2.50      0.09       824.89   93.25          38.09    4.34
std        1.12      0.09       554.65   86.56           8.36    0.38
min        1.00      0.00       126.00  -78.00          15.00    3.30
25%        1.75      0.00       416.00   35.75          33.00    4.07
50%        2.50      0.05       643.00   68.50          38.00    4.35
75%        3.00      0.10      1080.75  137.00          43.25    4.60
max        4.00      0.30      2600.00  366.00          65.00    5.00
```

Step 5: Clean and prepare the data

```python
# Check data quality issues
print("Missing values:")
print(df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
# Clean the dataset
df = df.drop_duplicates().copy()                        # remove duplicate rows
df["Customer_Type"] = df["Customer_Type"].fillna("Unknown")  # keep rows, label missing
# Create new analysis columns
df["Month_Num"] = df["Order_Date"].dt.month
df["Month"] = df["Order_Date"].dt.strftime("%b")
df["Discount_%"] = (df["Discount"] * 100).astype(int)
df["Delivery_Band"] = pd.cut(df["Delivery_Time"], bins=[0, 30, 45, 60, 100],
                             labels=["0-30 min", "31-45 min", "46-60 min", "60+ min"])
print("\nAfter cleaning -> shape:", df.shape)
print("Missing values left:", df.isnull().sum().sum())
```

Output:

```
Missing values:
Order_ID         0
Order_Date       0
Customer_ID      0
Customer_Type    6
City             0
Cuisine          0
Dish             0
Quantity         0
Discount         0
Order_Value      0
Profit           0
Delivery_Time    0
Rating           0
dtype: int64
Duplicate rows: 4
After cleaning -> shape: (200, 17)
Missing values left: 0
```

Step 6: Calculate key performance indicators (KPIs)

```
# Calculate key performance indicators (KPIs)
total_sales = df["Order_Value"].sum()
total_profit = df["Profit"].sum()
print(f"Total Revenue      : ₹{total_sales:,.0f}")
print(f"Total Profit       : ₹{total_profit:,.0f}")
print(f"Profit Margin      : {total_profit / total_sales * 100:.2f}%")
print(f"Total Orders       : {df['Order_ID'].nunique()}")
print(f"Average Order Value: ₹{df['Order_Value'].mean():,.0f}")
print(f"Loss-making Orders : {(df['Profit'] < 0).sum()}")
print(f"Avg Delivery Time  : {df['Delivery_Time'].mean():.1f} min")
print(f"Avg Customer Rating: {df['Rating'].mean():.2f} / 5")
```

Output:

```
Total Revenue      : ₹165,194
Total Profit       : ₹18,773
Profit Margin      : 11.36%
Total Orders       : 200
Average Order Value: ₹826
Loss-making Orders : 18
Avg Delivery Time  : 38.0 min
Avg Customer Rating: 4.35 / 5
```

Step 7: Exploratory data analysis

```
# Create a reusable summary function
def summarise(col):
    t = df.groupby(col, observed=True).agg(
        Orders=("Order_ID", "count"),
        Revenue=("Order_Value", "sum"),
        Profit=("Profit", "sum"),
        Avg_Rating=("Rating", "mean"),
    )
    t["Margin_%"] = (t["Profit"] / t["Revenue"] * 100).round(2)
    t["Avg_Rating"] = t["Avg_Rating"].round(2)
    return t[["Orders", "Revenue", "Profit", "Margin_%", "Avg_Rating"]]
print("--- By City ---")
print(summarise("City").sort_values("Profit", ascending=False))
print("\n--- By Cuisine ---")
print(summarise("Cuisine").sort_values("Profit", ascending=False))
print("\n--- By Dish ---")
print(summarise("Dish").sort_values("Profit", ascending=False))
print("\n--- By Discount (%) ---")
print(summarise("Discount_%").sort_index())
print("\n--- By Delivery Time ---")
print(summarise("Delivery_Band").sort_index())
```

Output:

```
--- By City ---
           Orders  Revenue  Profit  Margin_%  Avg_Rating
City
Delhi          61    54287    6525     12.02        4.37
Mumbai         59    47292    5157     10.90        4.32
Bengaluru      47    37722    4077     10.81        4.27
Hyderabad      33    25893    3014     11.64        4.46
--- By Cuisine ---
              Orders  Revenue  Profit  Margin_%  Avg_Rating
Cuisine
North Indian      44    53451    5531     10.35        4.35
Fast Food         50    55968    4364      7.80        4.29
Desserts          29    13046    2750     21.08        4.43
South Indian      37    16494    2380     14.43        4.36
Healthy           19    12767    1905     14.92        4.38
Chinese           21    13468    1843     13.68        4.28
--- By Dish ---
                     Orders  Revenue  Profit  Margin_%  Avg_Rating
Dish
Butter Chicken Combo      28    37700    3705      9.83        4.37
Pizza Large               29    40593    2691      6.63        4.27
Salad Bowl                19    12767    1905     14.92        4.38
Veg Hakka Noodles         21    13468    1843     13.68        4.28
Paneer Thali              16    15751    1826     11.59        4.32
Masala Dosa Combo         25    11550    1686     14.60        4.44
Burger Meal               21    15375    1673     10.88        4.33
Ice Cream Tub             14     7826    1422     18.17        4.37
Chocolate Brownie         15     5220    1328     25.44        4.49
Idli Vada Platter         12     4944     694     14.04        4.21
--- By Discount (%) ---
            Orders  Revenue  Profit  Margin_%  Avg_Rating
Discount_%
0               65    59270    9874     16.66        4.32
5               53    42647    6055     14.20        4.37
10              34    26748    2324      8.69        4.35
20              34    29144     628      2.15        4.33
30              14     7385    -108     -1.46        4.41
--- By Delivery Time ---
              Orders  Revenue  Profit  Margin_%  Avg_Rating
Delivery_Band
0-30 min          37    30846    3712     12.03        4.63
31-45 min        127   105548   13480     12.77        4.36
46-60 min         35    28344    1521      5.37        4.03
60+ min            1      456      60     13.16        3.50
```

```python
# Check the relationship between numeric columns
cols = ["Quantity", "Discount", "Order_Value", "Profit", "Delivery_Time", "Rating"]
print(df[cols].corr().round(2))
```

Output:

```
               Quantity  Discount  Order_Value  Profit  Delivery_Time  Rating
Quantity           1.00      0.06         0.68    0.43          -0.07    0.05
Discount           0.06      1.00        -0.12   -0.62          -0.07    0.03
Order_Value        0.68     -0.12         1.00    0.50           0.01   -0.04
Profit             0.43     -0.62         0.50    1.00          -0.14    0.08
Delivery_Time     -0.07     -0.07         0.01   -0.14           1.00   -0.60
Rating             0.05      0.03        -0.04    0.08          -0.60    1.00
```

```python
# Extra checks used for the insights table
share = df.groupby("Cuisine")["Order_Value"].sum() / total_sales * 100
print("Revenue share by cuisine (%):\n", share.round(1))
high = df[df["Discount"] >= 0.20]
print("\nOrders with 20%+ discount:", len(high),
      "| revenue share:", round(high["Order_Value"].sum() / total_sales * 100, 1), "%",
      "| profit: ₹", high["Profit"].sum())
print("Loss-making orders from 20%+ discount:", (high["Profit"] < 0).sum(),
      "of", (df["Profit"] < 0).sum())
late = df[df["Delivery_Time"] > 45]
print("\nLate orders (>45 min):", len(late), f"({len(late) / len(df) * 100:.1f}% of orders)")
print("Avg rating  late vs on-time:", round(late["Rating"].mean(), 2),
      "vs", round(df[df["Delivery_Time"] <= 45]["Rating"].mean(), 2))
q = df.groupby(["Month_Num", "Month"])["Order_Value"].sum().sort_index(level=0)
print("\nTop 3 months by revenue:", q.sort_values(ascending=False).head(3).droplevel(0).to_dict())
```

```
print("Oct-Dec revenue share:", round(df[df["Month_Num"] >= 10]["Order_Value"].sum() / total_sales * 100, 1), "%")
```

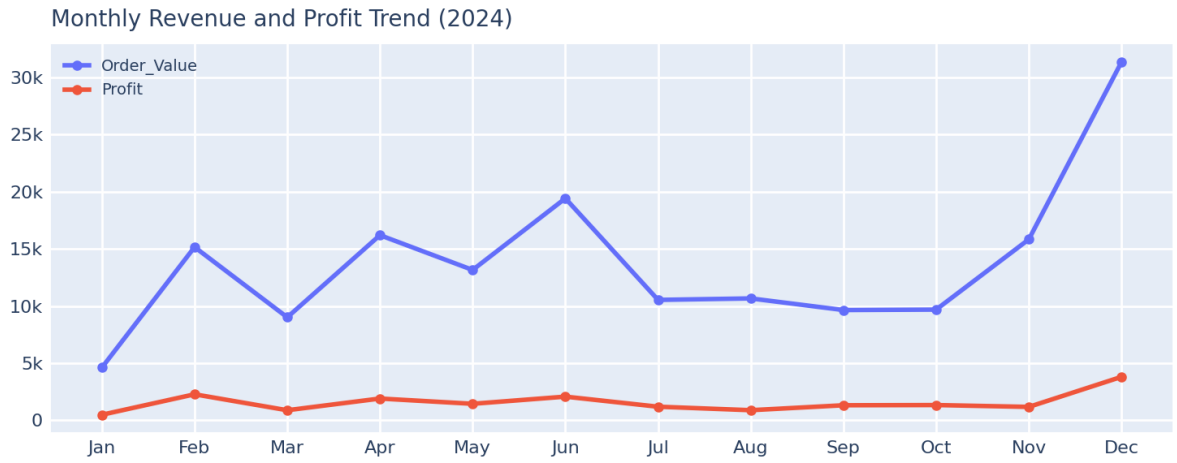
Output:

```
Revenue share by cuisine (%):
 Cuisine
Chinese          8.2
Desserts         7.9
Fast Food       33.9
Healthy          7.7
North Indian    32.4
South Indian    10.0
Name: Order_Value, dtype: float64
Orders with 20%+ discount: 48 | revenue share: 22.1 % | profit: ₹ 520
Loss-making orders from 20%+ discount: 16 of 18
Late orders (>45 min): 36 (18.0% of orders)
Avg rating  late vs on-time: 4.02 vs 4.42
Top 3 months by revenue: {'Dec': 31333, 'Jun': 19410, 'Apr': 16191}
Oct-Dec revenue share: 34.4 %
```

Step 8: Visual exploration with Plotly

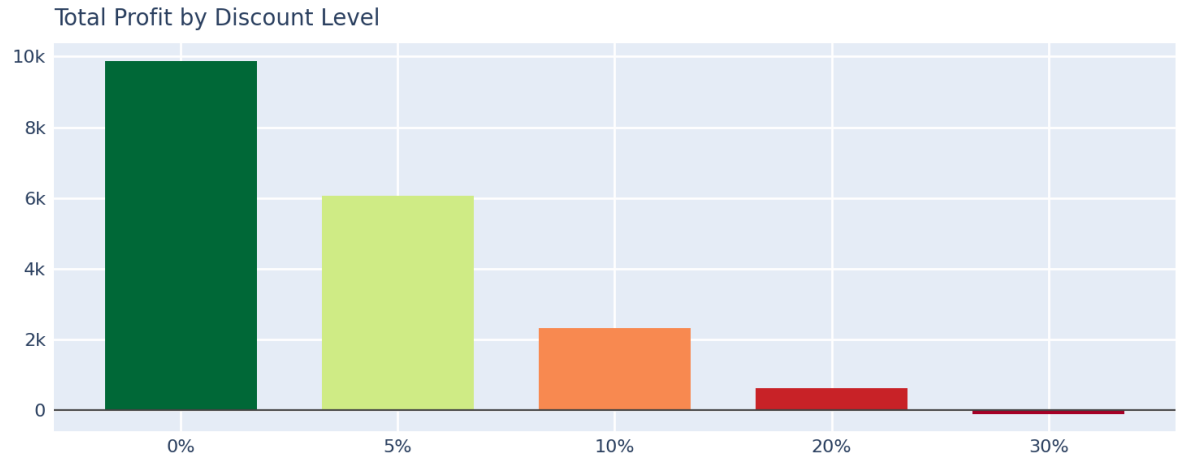
```
# Monthly revenue and profit trend
monthly = (df.groupby(["Month_Num", "Month"], as_index=False)[["Order_Value", "Profit"]]
             .sum().sort_values("Month_Num"))
fig = px.line(monthly, x="Month", y=["Order_Value", "Profit"], markers=True,
              title="Monthly Revenue and Profit Trend (2024)")
fig.show()
```

Output:



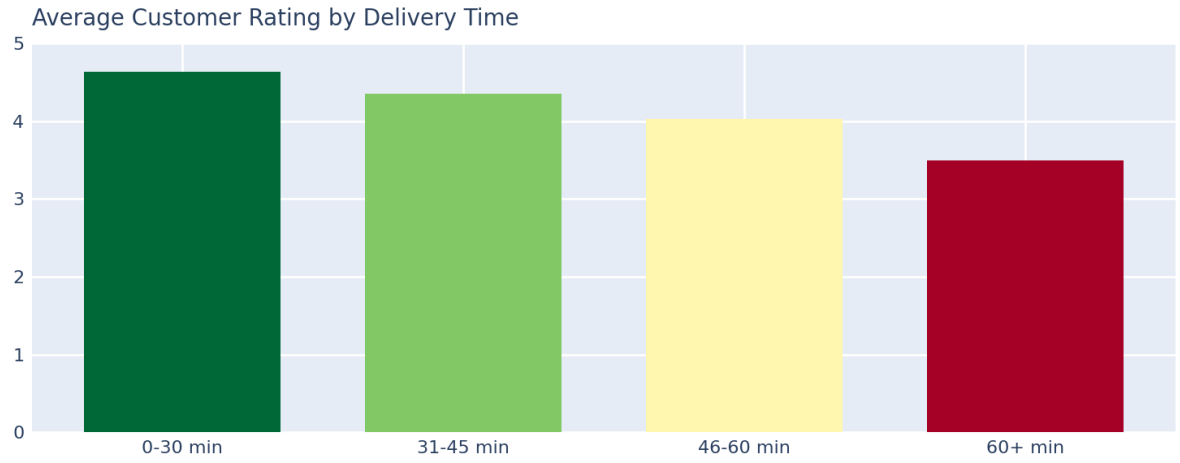

```
# Impact of discount on profit
disc = df.groupby("Discount_%", as_index=False)["Profit"].sum()
disc["Discount_%"] = disc["Discount_%"].astype(str) + "%"
fig = px.bar(disc, x="Discount_%", y="Profit", color="Profit",
             color_continuous_scale="RdYlGn",
             title="Total Profit by Discount Level")
fig.show()
```

Output:




```
# Impact of delivery time on customer rating
band = df.groupby("Delivery_Band", observed=True, as_index=False)["Rating"].mean()
fig = px.bar(band, x="Delivery_Band", y="Rating", color="Rating",
             color_continuous_scale="RdYlGn", range_y=[0, 5],
             title="Average Customer Rating by Delivery Time")
fig.show()
```

Output:



Step 9: Create the dashboard layout

```
# Helper function to build a KPI card
def kpi_card(title, value, color):
    return html.Div([
        html.P(title, style={"margin": "0", "fontSize": "14px", "color": "#555"}),
        html.H2(value, style={"margin": "5px 0 0 0", "color": color}),
    ], style={"flex": "1", "padding": "15px", "borderRadius": "8px",
              "backgroundColor": "#f4f6f9", "borderLeft": f"6px solid {color}"})
# Create a Dash application
app = Dash(__name__)
# Design the dashboard
app.layout = html.Div([
    # Add title
    html.H1(
        "Food Delivery Performance Dashboard",
        style={"textAlign": "center", "color": "blue"}
    ),
    # Create the filters (two dropdowns)
    html.Div([
        html.Div([
            html.Label("Select City:"),
            dcc.Dropdown(
                id="city",
                options=[{"label": "All Cities", "value": "All"}] + [
                    {"label": c, "value": c}
                    for c in sorted(df["City"].unique())
                ],
                value="All",
                clearable=False
            ),
        ], style={"width": "48%", "display": "inline-block"}),
        html.Div([
            html.Label("Select Cuisine:"),
            dcc.Dropdown(
                id="cuisine",
                options=[{"label": "All Cuisines", "value": "All"}] + [
                    {"label": c, "value": c}
                    for c in sorted(df["Cuisine"].unique())
                ],
                value="All",
                clearable=False
            ),
        ], style={"width": "48%", "display": "inline-block",
                  "marginLeft": "4%"}),
    ]),
    # Area for KPI cards
    html.Div(id="kpi_cards",
             style={"display": "flex", "gap": "15px", "margin": "20px 0"}),
    # Add charts
    dcc.Graph(id="trend_chart"),
    dcc.Graph(id="dish_chart"),
    dcc.Graph(id="discount_chart"),
    dcc.Graph(id="customer_chart"),
    # Ethical note for the viewer
    html.P("Note: Data is simulated. No personal information is used. "
           "Insights support, but do not replace, human decisions.",
           style={"fontSize": "12px", "color": "gray"})
], style={"padding": "20px"})
```

Step 10: Add interactive dashboard functionality

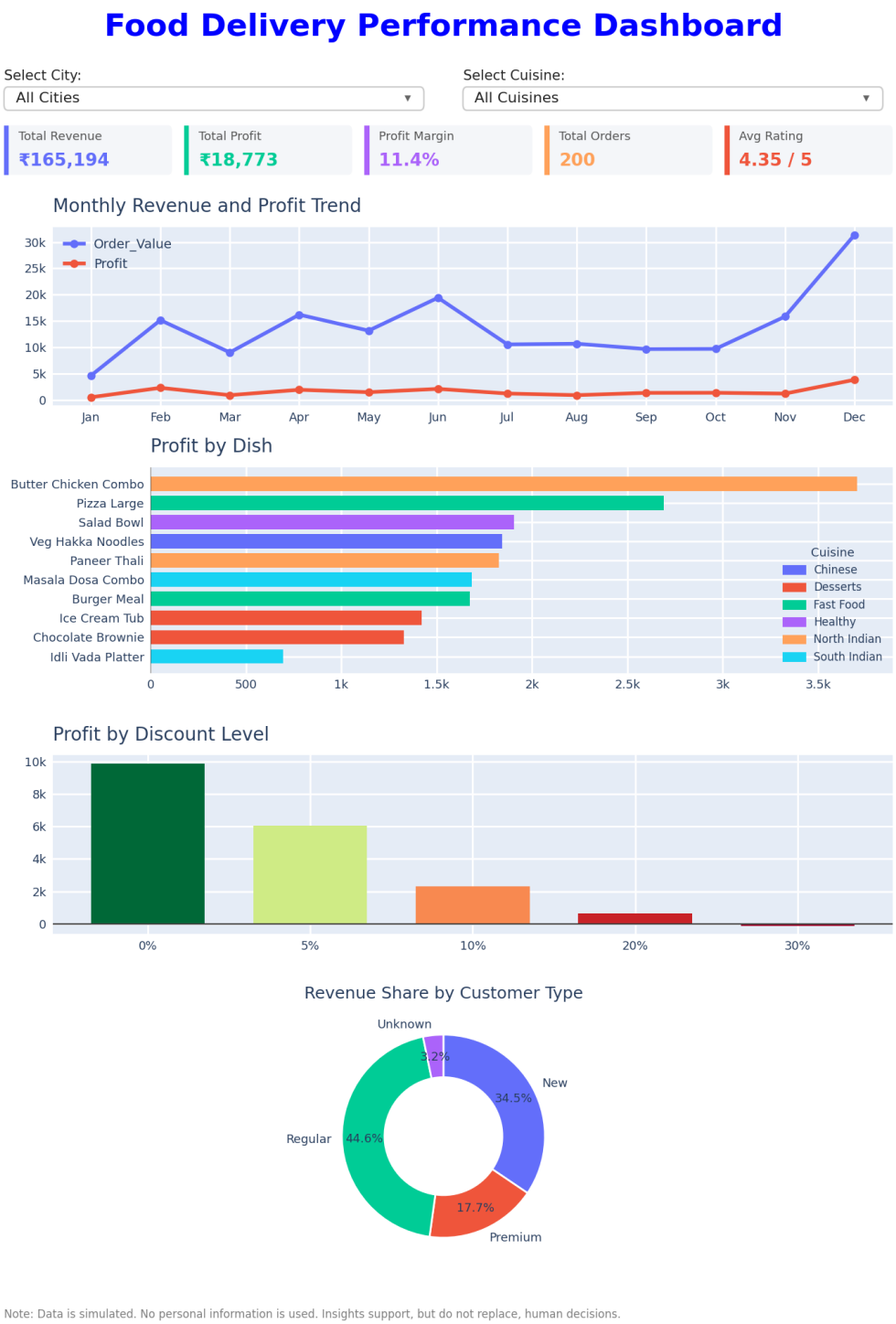
```
# Update the dashboard
@app.callback(
    Output("kpi_cards", "children"),
    Output("trend_chart", "figure"),
    Output("dish_chart", "figure"),
    Output("discount_chart", "figure"),
    Output("customer_chart", "figure"),
    Input("city", "value"),
    Input("cuisine", "value")
)
# Define the function
def update_dashboard(city, cuisine):
```

```
    # Filter the dataset
    filtered = df.copy()
    if city != "All":
        filtered = filtered[filtered["City"] == city]
    if cuisine != "All":
        filtered = filtered[filtered["Cuisine"] == cuisine]
    # Calculate KPIs
    revenue = filtered["Order_Value"].sum()
    profit = filtered["Profit"].sum()
    margin = profit / revenue * 100 if revenue else 0
    avg_rating = filtered["Rating"].mean() if len(filtered) else 0
    cards = [
        kpi_card("Total Revenue", f"₹{revenue:,.0f}", "#636EFA"),
        kpi_card("Total Profit", f"₹{profit:,.0f}", "#00CC96"),
        kpi_card("Profit Margin", f"{margin:.1f}%", "#AB63FA"),
        kpi_card("Total Orders", f"{len(filtered)}", "#FFA15A"),
        kpi_card("Avg Rating", f"{avg_rating:.2f} / 5", "#EF553B"),
    ]
    # Create a line chart (monthly trend)
    monthly = (filtered.groupby(["Month_Num", "Month"], as_index=False)
               [["Order_Value", "Profit"]].sum().sort_values("Month_Num"))
    trend_fig = px.line(monthly, x="Month", y=["Order_Value", "Profit"],
                        markers=True, title="Monthly Revenue and Profit Trend")
    # Create a horizontal bar chart (profit by dish)
    dsh = (filtered.groupby(["Dish", "Cuisine"], as_index=False)["Profit"]
           .sum().sort_values("Profit"))
    dish_fig = px.bar(dsh, x="Profit", y="Dish", color="Cuisine",
                      orientation="h", title="Profit by Dish")
    # Create a bar chart (profit by discount level)
    disc = filtered.groupby("Discount_%", as_index=False)["Profit"].sum()
    disc["Discount_%"] = disc["Discount_%"].astype(str) + "%"
    discount_fig = px.bar(disc, x="Discount_%", y="Profit", color="Profit",
                          color_continuous_scale="RdYlGn",
                          title="Profit by Discount Level")
    # Create a donut chart (revenue share by customer type)
    customer_fig = px.pie(filtered, names="Customer_Type", values="Order_Value",
                          hole=0.4, title="Revenue Share by Customer Type")
    return cards, trend_fig, dish_fig, discount_fig, customer_fig
```

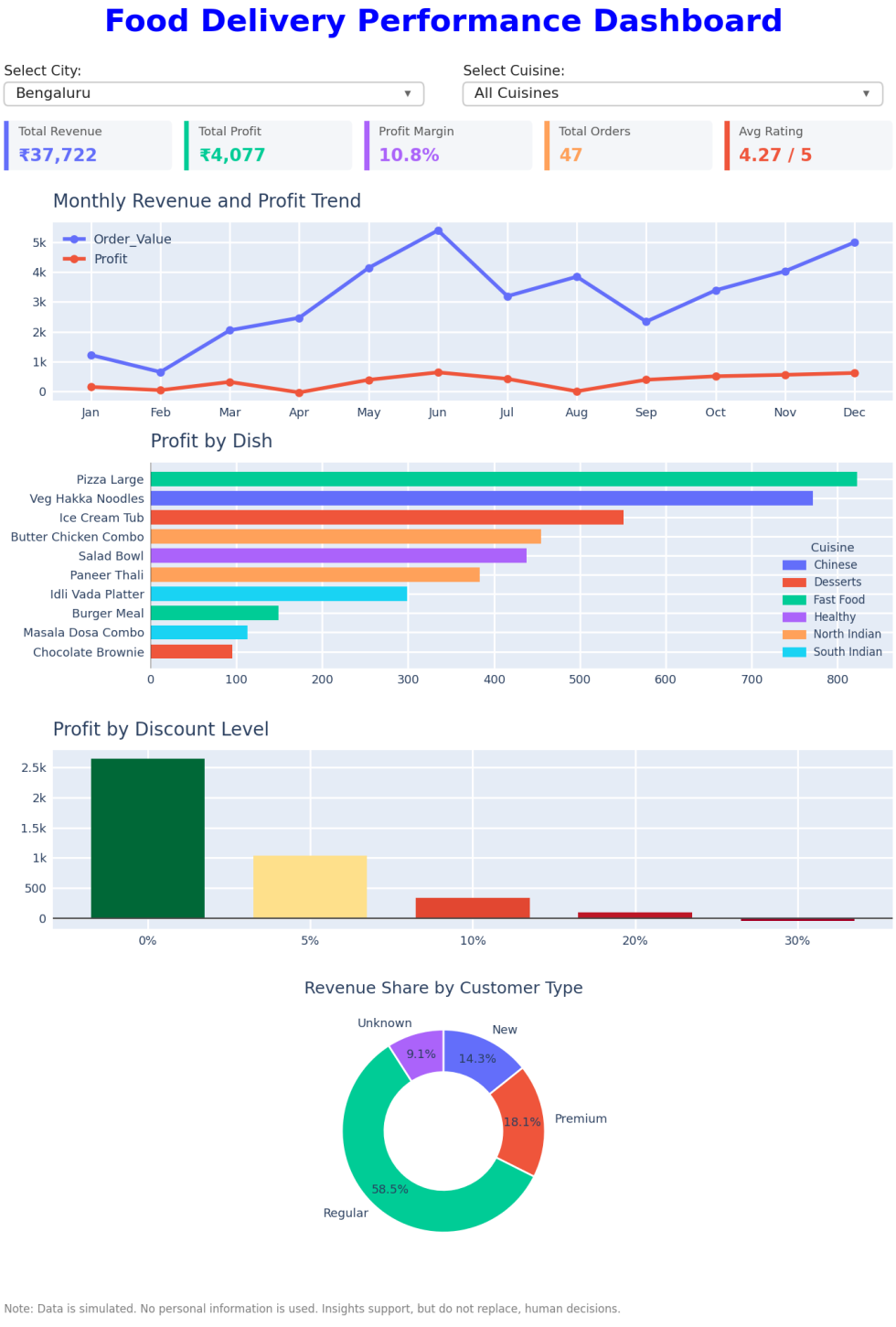
Step 11: Run the dashboard in Google Colab

```
# Run the dashboard
app.run(jupyter_mode="inline", debug=False, port=8050)
```

Output:



Dashboard output: City = All Cities, Cuisine = All Cuisines



Dashboard output after selecting City = Bengaluru (all charts and KPIs update)

Actionable Insights

#	Insight (evidence from the data)	Recommended action
1	Fast Food and North Indian bring 66.3% of revenue but only 52.7% of profit (margins 7.8% and 10.4%). Pizza Large has the highest revenue (₹40,593) but the lowest margin (6.6%). Desserts (21.1%), Healthy (14.9%) and South Indian (14.4%) earn far more per rupee.	Protect volume of Pizza and Butter Chicken but negotiate restaurant commission and packaging costs. Promote Desserts and Healthy items as add-ons to lift overall margin.
2	Orders with 20% or 30% discount (48 orders, 22.1% of revenue) earned only ₹520 in total profit, and 16 of the 18 loss-making orders (₹548 of ₹591 lost) came from these two levels. Full-price orders earn a 16.7% margin. Discounts do not improve ratings (4.32 at 0% vs 4.33 at 20%).	Cap discounts at 10%; allow 20%+ offers only for new-customer acquisition, with a fixed budget and manager approval.
3	Late orders (over 45 minutes) are 18.0% of all orders and earn only a 5.5% margin against 12.6% for on-time orders. Their average rating is 4.02 vs 4.42, and delivery time has the strongest link with rating (correlation -0.60).	Set a 45-minute delivery target; add rider capacity at peak hours and review kitchen preparation time before blaming riders.
4	Bengaluru has the lowest margin (10.8% vs 11.5% elsewhere) and the slowest delivery (average 41.7 min). 34.0% of its orders are late, against 13.1% in the other cities.	Add riders and redraw delivery zones in Bengaluru before spending on new marketing there.
5	December is the peak month (₹31,333, 19.0% of revenue) and October to December bring 34.4% of annual revenue; June is a second peak (₹19,410). January is the weakest month (₹4,610) and November has the lowest margin (7.4%) despite high volume.	Plan rider shifts and restaurant-partner capacity early for the festive season; avoid deep discounts in November and use small, targeted offers in January.

Ethical Considerations

Area	How it is addressed in this project
Privacy & data protection	The dataset is simulated and holds no names, phone numbers, emails or delivery addresses. Customer IDs are anonymised. With real data, addresses and order history are personal data and must be handled under India's Digital Personal Data Protection Act, 2023, using only what is needed (data minimisation).
Consent & purpose limitation	Food orders can reveal habits, diet, religion or health (for example fasting-day or diabetic-friendly orders). Real customer data should be used only for service and business reporting, not re-used for profiling or targeting without consent.
Fairness & bias	City and customer-type comparisons must not be used to give worse prices, longer waits or fewer offers to an area or group of people. Orders with a missing customer type were kept as 'Unknown' instead of being silently dropped.
Delivery partner welfare	Delivery-time targets must not push riders towards unsafe speeds. Late-delivery data should guide staffing and process fixes, not punish individual riders without checking traffic, rain and kitchen delays.
Transparency of cleaning	Every cleaning step is shown in the notebook: 4 duplicate rows removed, 6 missing values labelled 'Unknown'. Nothing was changed silently.
Honest visualisation	Bar charts start at zero, axes are labelled, colours show meaning (red for loss, green for profit) and the dashboard states that the data is simulated.
Correlation is not causation	The links between high discounts and low profit, and between slow delivery and low ratings, support a policy review, not automatic decisions. Other factors (weather, restaurant preparation time, competition) must be checked.
Security & access	A real dashboard needs login, role-based access and no public sharing of sales data. Here the app runs only inside the notebook session.
Human accountability	The dashboard supports decision-makers; people remain responsible for decisions such as discount policy, rider staffing or targets.

Limitations

- The data is simulated for one year (2024); real data would include cancellations, refunds, rider costs, marketing spend and customer history.
- Profit is analysed per order; fixed costs such as technology, salaries and rider incentives are not included.
- With only 200 orders, month-wise and city-wise figures are noisy; small differences should not be over-read.
- Ratings here depend mainly on delivery time; real ratings also depend on food quality and packaging.
- Findings should be validated on real data before business use.

Conclusion

An interactive Dash/Plotly dashboard was built on a cleaned food delivery dataset. The dropdown filters for City and Cuisine update the KPI cards and four charts instantly, helping managers explore performance without writing code. The analysis shows that discount control, faster delivery (especially in Bengaluru) and festive and monsoon season planning are the main opportunities, and the ethical checks (privacy, fairness, rider welfare, transparency and human oversight) ensure the insights are used responsibly.